

# Price comes last: gating an AI model change on accuracy

*How I set the bar for an AI model change on an internal knowledge product — and why cost was the last thing we looked at.*

---

## The decision, up front

A cheaper model came up for the part of our product that turned meeting transcripts into structured notes and tasks. On paper it was an easy yes: same family of capability, materially lower cost per call, a drop-in swap.

I said we weren't switching until it cleared an accuracy bar against a fixed benchmark — and that we would not look at the price until it had.

That rule outlasted the question that prompted it. It became the standing process for every prompt and model change on the product afterwards.

This is the reasoning behind it, because the reasoning is the part that transfers.

---

## The situation

I was product owner for an internal knowledge and meeting-notes product at a small AI startup, and a contributor to its codebase. The team it served was small and cross-functional — engineering, product and design — and ran on daily standups plus a scatter of project calls.

The product took meeting transcripts, generated notes with the context of everything that had come before, and created tasks from what was actually discussed. A reviewer approved those tasks before anything was pushed into our task tracker. That approval gate matters to the rest of this story.

## The problem worth solving

We already had meeting notes before I owned this. They were useless, and the reason was structural.

They were accurate. Each note was a fair summary of the meeting it came from. But each one arrived with no memory — no previous discussion, no project history, no idea who was already working on what. A note that says "the team discussed the ingestion issue" is technically correct and operationally worthless when the ingestion issue has been discussed in four previous meetings and is already assigned.

The cost showed up as things falling through. Work got lost between meetings. Decisions got re-litigated because nobody could find where they'd been made. Tasks were mentioned aloud, agreed to, and never written down, so they existed only in whoever happened to remember.

So the product wasn't a summarizer. It was a continuity problem wearing a summarizer's clothes. That reframing decided everything downstream: if the job is continuity, then the thing you must never get wrong is **who owes what to whom**. A note that reads a little awkwardly is a minor irritation. A task assigned to the wrong person, or silently dropped, breaks the thing the product exists to do.

## The decision point

Later, a cheaper model became available for the note-and-task generation step. The case for switching was straightforward and mostly correct: it was meaningfully cheaper per call, generation ran on every meeting

every day, and this was one of our recurring AI costs rather than a one-off. Nobody was being reckless by proposing it.

The reason I pushed back wasn't that the model was bad. Nobody knew whether it was bad. That was the problem.

We had no way to answer the question. "Does the output look fine?" was the whole evaluation method available, and for this product that method is actively misleading. Notes from a slightly worse model still *read* fine. Fluency is the easiest thing for a model to keep and the last thing it loses. What degrades first is the unglamorous structural work — noticing that an action item was assigned, catching that two mentions are the same task, not inventing an owner for something nobody claimed.

None of that is visible by reading a sample output and nodding. You would find out later, from a person who never got told about something they were supposed to do, and you would have no way of connecting that back to a model change made weeks before.

## How I framed the call

I set two rules.

**First: a fixed benchmark, built before we evaluate anything.** We assembled a set of past meetings, chosen to span the kinds of meetings we actually had, from short standups to longer planning calls, where we already knew what the correct output was. Every candidate model ran against the same set. Not a fresh sample each time, not whatever meetings happened that week. The same set, every time, so results were comparable across months and across changes.

**Second: accuracy gates before cost is considered.** Not "weigh accuracy against cost" — gate. A model that failed the accuracy bar was out, and its price was never discussed. Only models that cleared the bar were compared on cost.

That ordering was the actual decision, and it's the part I'd defend hardest. Weighing quality against cost sounds more balanced and more grown-up, and in practice it is how you talk yourself into a worse product. When both numbers are on the table at once, the cost figure is precise, immediate and legible to everyone, while the quality figure is noisy and contested. The precise number wins arguments against the fuzzy one, every time, regardless of which one matters more. Sequencing them removes that failure mode structurally rather than relying on people to be disciplined in the moment.

We scored on four things, chosen because each maps to a specific way the product fails a real person:

| What we measured          | The failure it catches                                                      |
|---------------------------|-----------------------------------------------------------------------------|
| <b>Missed actions</b>     | Something was agreed and never became a task                                |
| <b>Duplicates</b>         | The same task created repeatedly across meetings, so the list becomes noise |
| <b>Coverage</b>           | Topics discussed but absent from the notes entirely                         |
| <b>Ownership accuracy</b> | A task assigned to the wrong person, or to nobody                           |

Ownership accuracy was the one I cared most about, and it's the one a generic quality score would have buried. A model can score well overall while being unreliable at exactly the thing this product exists to get right.

## What the rule was built to catch, and what I chose not to do

The failure the benchmark existed to catch is a model that is fine at producing readable notes and worse at the structural work: exactly what eyeballing a few outputs would miss. A candidate either clears the bar on

all four numbers or it doesn't reach the price conversation.

Three things I deliberately didn't do:

**I didn't build a general evaluation platform.** It was a scripted run against a fixed set with four metrics. There was a real pull toward building something more configurable and reusable, and it would have been more satisfying to build. It also would have delayed the decision to produce a better version of an answer we could already get.

**I didn't chase a single quality score.** One number would have been easier to report and easier to argue about. It would also have let a model trade ownership accuracy for fluency and still look like an improvement. Four separate numbers meant a regression in the thing that mattered stayed visible instead of averaging away.

**I didn't remove the human approval gate,** even though better model scores were the obvious argument for removing it. The gate was what made a bad generation recoverable instead of silently wrong, and I wasn't willing to trade a recoverable failure for an invisible one to save reviewer time.

## What it changed

The benchmark outlived the decision it was built for.

It became the precondition for any prompt or model change on that product: run the set, show the four numbers, then we talk. That turned a category of argument — someone believes the new prompt is better, someone else doesn't — into something checkable in an afternoon. Individual model choices will keep changing as the landscape moves. The rule for making them doesn't have to.

The wider lesson: the durable artifact from an AI product decision is usually not the decision. It's the apparatus that let you make it, because you will face the same question again in three months with different models.

## Next time: build the benchmark first

The benchmark here was assembled because a decision was waiting on it. That works, and it has a cost: a set built with one question in front of it is shaped a little by that question, rather than by the product's failure modes in general, and a benchmark built while a decision waits is one somebody is impatient with.

The version I'd run from the start: build the evaluation set early, while nothing hangs on it, and treat it as part of shipping the feature rather than as a thing you reach for when a choice arrives. It costs about the same amount of work and produces a more honest instrument — one that describes the product, rather than one shaped by the first argument it had to settle.

---

*I work on AI products end to end — deciding what ships and building the systems underneath. This is one of a series on product decisions in AI systems.*